

Baby Cry Classification System: Technical Review Report

Prepared for Discussion

Focus Areas: Data Collection, Notebook Preprocessing (`Production_Work (1).ipynb`), Production API Preprocessing, and Server Deployment.

This report provides a comprehensive review of the crying sound classification model within the `BabyCry` project. It traces the lifecycle of the audio data from collection and augmentation through the training pipeline in the `Production_Work` notebook, and details the final deployment architecture for serving the model via the FastAPI backend.

1. Data Collection and Preparation

The foundation of the classification model relies on aggregating and refining diverse audio datasets to create a robust and generalized system.

Data Sources (As seen in the `Production_Work` Notebook)

The training data was aggregated programmatically (via the Kaggle API and direct downloads) from multiple repositories to ensure acoustic diversity:

- **Kaggle Datasets:** Primary cry data (`baby_crying`) and mixed vocalizations (`baby_sounds`).
- **Donate-a-Cry (infant_cry_corpus):** Secondary annotated cry recordings.
- **ESC-50:** Environmental sounds used as the negative class ("not_baby_cry") for Stage 1.
 - *Crucial Data Cleaning.* The notebook explicitly drops ESC-50 categories that overlap with human vocalizations (e.g., `crying_baby` , `laughing` , `sneezing` , `breathing` , `coughing` , `snoring`) to prevent confusing the model during binary "cry vs. not-cry" training.

Label Consolidation Strategy

A crucial engineering decision was merging the original highly-granular labels into **4 clinically-meaningful categories** to improve classification stability:

- `hungry` + `tired` + `discomfort` + `cold/hot` → **`needs`**
- `belly_pain` → **`physical_pain`**
- `scared` → **`scared`** (preserved)
- `burping` → **`burping`** (preserved)

[!NOTE] The dataset was split into **80% Training/Validation (Pool)** and **20% Testing** using stratified sampling (stratified by the original fine-grained labels) to ensure proportional class distribution and diversity across all splits.

2. Preprocessing Stages: Training vs. Production

The project carefully distinguishes between "offline" training preprocessing (in the notebook) and "online" fast preprocessing (in the API).

Training Preprocessing (`Production_Work (1).ipynb`)

During the dataset preparation phase in the notebook, the audio goes through a heavy, production-grade cleaning pipeline:

- **Decoding & Resampling:** Loaded using `librosa` , forced to mono, and resampled to **16,000 Hz**.
- **High-Pass Filtering:** Applied a 50 Hz Butterworth high-pass filter (order 4) to remove low-frequency rumble and DC offset.
- **Loudness Normalization:** Normalized to **-23.0 LUFS** (broadcast standard) using `pyloudnorm` so the model doesn't overfit to recording volume.
- **VAD Trimming:** WebRTC Voice Activity Detection (VAD, aggressiveness level 2) was used to dynamically trim leading and trailing silence from the cry recordings.

Inference Preprocessing (`inference/inference.py` & `api/`)

In the deployed server, the preprocessing is optimized for speed and safety:

1. **File Validation & Acoustic Sanity Checks:** Before the model even sees the audio, the API checks file size/duration limits and runs fast acoustic tests (RMS energy > 0.010, Voiced ratio > 20%, proper cry-band frequency ratios) to instantly reject pure silence, static, or constant tones.
2. **Audio Transformation:**
 - Decoded via `soundfile` for speed.
 - Resampled to **16 kHz**.
 - **Fixed Duration Padding/Cropping:** Fixed to exactly **5 seconds (80,000 samples)**. Longer files are center-cropped; shorter files are zero-padded with a binary attention mask.
 - **Z-Score Normalization:** Per-sample zero-mean and unit-variance normalization, which exactly matches the expected input distribution of the HuBERT backbone.

3. Core Code Analysis: `Production_Work (1).ipynb`

The Jupyter notebook `Production_Work (1).ipynb` is the master orchestration file for the AI model. Its core pipeline consists of:

1. **Two-Stage Cascade Architecture:**
 - Built on top of `facebook/hubert-base-1s960` using PyTorch Lightning.

- The feature extractor CNN of HuBERT is frozen, and a custom classification head (with GELU, Dropout, and masked mean pooling) is added on top.
 - **Stage 1**: Binary Classification (Baby Cry vs. Not Baby Cry).
 - **Stage 2**: 4-Class Classification (Needs, Physical Pain, Scared, Burping).
2. **Handling Class Imbalance**: The notebook computes and injects inverse-frequency class weights directly into the CrossEntropy loss function (e.g., heavily weighting the rare `burping` class and down-weighting the abundant `needs` class).
 3. **Ensemble & Thresholds**: For Stage 2, it evaluates multiple checkpoints, implements an ensemble averaging technique across epochs 6 and 7, and optimizes confidence thresholds to abstain from low-confidence predictions.
 4. **Packaging**: At the very end of the notebook, it programmatically copies the best PyTorch Lightning checkpoints, generates the configuration JSONs, and literally writes out the `inference.py` script to a `backend_package_v1` directory, ensuring that the inference code deployed to the server perfectly matches the training environment.

4. Uploading the Model to the Server (Deployment)

The deployment architecture (`api/main.py`) takes the packaged outputs from the notebook and wraps them in a production-ready API.

Server Architecture

- **Framework**: Built on **FastAPI** running on a **Uvicorn** ASGI server for high-performance asynchronous request handling.
- **Endpoints**: Exposes a `POST /predict` endpoint that accepts `multipart/form-data` audio uploads.
- **In-Process Singleton Serving**:
 - The system loads all 3 HuBERT checkpoints (~3.3 GB total) directly into memory exclusively during the FastAPI `@app.on_event("startup")` lifecycle hook. It even runs a dummy "warm-up" inference array to trigger PyTorch JIT compilation, ensuring the very first user request is fast.
 - This eliminates per-request model loading overhead, achieving GPU inference times of ~150ms.

Production Hardening Features

- **Rate Limiting**: Integrated `slowapi` to enforce limits of 30 requests/minute and 500 requests/hour per IP address.
- **Stateless Design**: Audio is processed entirely in-memory (`io.BytesIO`) and immediately discarded. No audio data is persisted on the server.
- **Structured Logging & Metrics**: Uses `structlog` for machine-parseable JSON logging (ideal for Datadog/ELK) and exposes a `/health` endpoint to monitor uptime, request counts, inference times, and acoustic rejections.
- **Arabic Localization Engine**: The server dynamically interprets the model's raw probability outputs, mapping them to localized Arabic content (parenting tips, physical signs, and a strict medical disclaimer) via `api/content.py`.

Containerization Strategy

The system is fully containerized with a `Dockerfile` and `docker-compose.yml`. It uses a `python:3.10-slim` image, pre-installs `libsndfile1` (required by `soundfile`), and runs the Uvicorn server on port 8501.

[!TIP] **Discussion Point for Scalability**: The current system requires ~5 GB of memory because it loads 3 separate PyTorch Lightning checkpoints. For horizontal scaling, we could discuss exporting the models to **ONNX format**, or modifying the architecture so Stage 1 and Stage 2 share the same 95M-parameter HuBERT backbone in memory, reducing the memory footprint by over 60%.